TALLER I

CORTE I

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

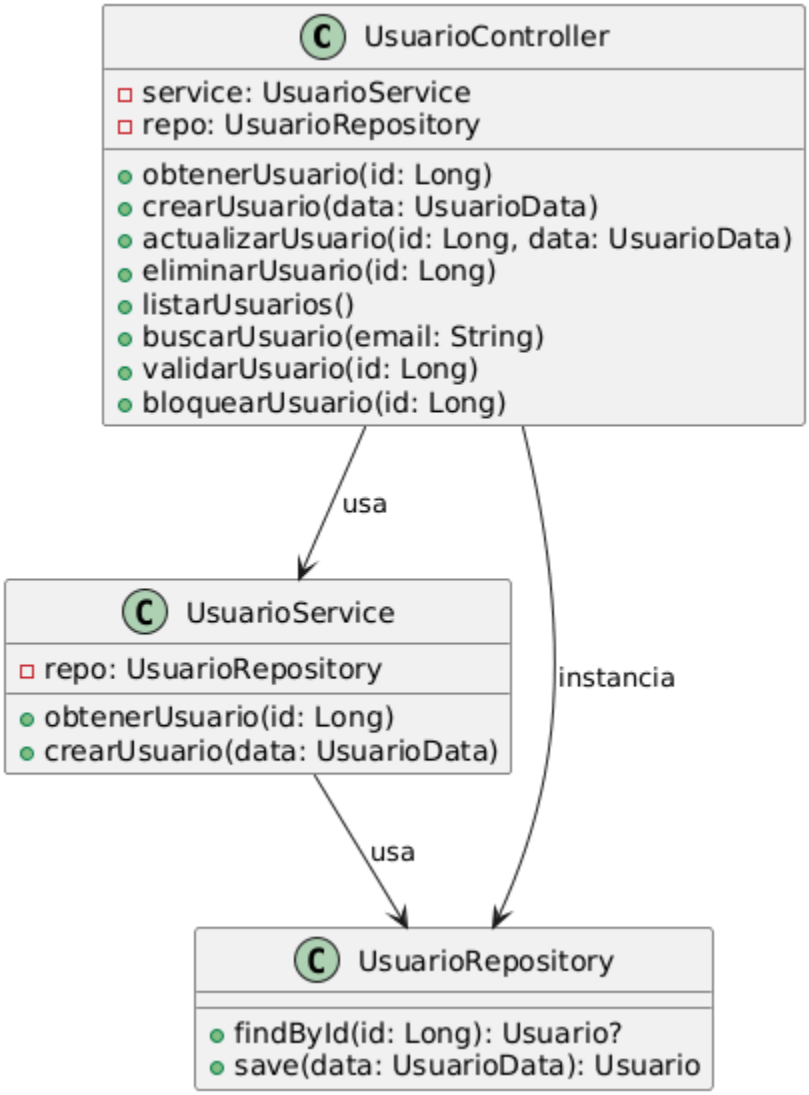
JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

EJERCICIO DE CLASE (CODIGO)

```
class UsuarioController {
 private val service = UsuarioService()
 private val repo = UsuarioRepository()

 fun obtenerUsuario(id: Long) = service.obtenerUsuario(id)
 fun crearUsuario(data: UsuarioData) = service.crearUsuario(data)
 fun actualizarUsuario(id: Long, data: UsuarioData) =
 service.actualizarUsuario(id, data)
 fun eliminarUsuario(id: Long) = service.eliminarUsuario(id)
 fun listarUsuarios() = service.listarUsuarios()
 fun buscarUsuario(email: String) = service.buscarUsuario(email)
 fun validarUsuario(id: Long) = service.validarUsuario(id)
 fun bloquearUsuario(id: Long) = service.bloquearUsuario(id)
}

class UsuarioService {
 private val repo = UsuarioRepository()

 fun obtenerUsuario(id: Long) = repo.findById(id)
 fun crearUsuario(data: UsuarioData) = repo.save(data)
}

class UsuarioRepository {
 fun findById(id: Long): Usuario? = null
 fun save(data: UsuarioData): Usuario = Usuario()
}
```

1. DIAGRAMA DE CLASES UML



2. RESPONSABILIDADES DE CADA CLASE

2.1 UsuarioController

Responsabilidad principal:

Actuar como intermediario entre la capa de presentación y la capa de negocio.

Responsabilidades secundarias:

- Recibir las solicitudes relacionadas con la entidad Usuario.
- Delegar las operaciones al servicio.

- Exponer métodos para las operaciones CRUD.
- No debe contener lógica de negocio.

2.2 UsuarioService

Responsabilidad principal:

Gestionar la lógica de negocio relacionada con la entidad Usuario.

Responsabilidades secundarias:

- Implementar reglas de negocio asociadas a los usuarios.
- Coordinar operaciones entre el controlador y el repositorio.
- Validar datos.
- Gestionar procesos como creación y consultas.
- Utilizar el repositorio para el acceso a datos.

2.3 UsuarioRepository

Responsabilidad principal:

Gestionar el acceso y la persistencia de datos relacionados con la entidad Usuario.

Responsabilidades secundarias:

- Consultar usuarios en la base de datos.
- Guardar información de usuarios.
- Encapsular la lógica de acceso a datos.
- Servir como intermediario entre la base de datos y la capa de servicio.

3. IDENTIFICACIÓN DE CODE SMELLS

3.1 Acoplamiento Fuerte (Violación del Principio de Inversión de Dependencias)

```
private val service = UsuarioService()

private val repo = UsuarioRepository()
```

Esto genera:

- Alto acoplamiento entre clases.
- Dificultad para realizar pruebas unitarias.
- Imposibilidad de aplicar inyección de dependencias.
- Violación del principio SOLID de Inversión de Dependencias.

Antipatrón: **CARGO CULT PROGRAMMING**

Este antipatrón ocurre cuando se copia una estructura (por ejemplo Controller–Service–Repository) sin comprender realmente sus principios y responsabilidades unicas.

3.2 Atributo No Utilizado

En la clase UsuarioController se declara un objeto UsuarioRepository que no es utilizado.

Esto representa:

- Código innecesario.
- Baja cohesión.
- Posible error de diseño o refactorización incompleta.

Antipatrón: LAVA FLOW

Lava Flow ocurre cuando se deja código muerto, innecesario o restos de implementaciones anteriores por miedo a eliminarlo o por descuido.

3.3 Inconsistencia en la Capa de Servicio

El controlador define múltiples métodos (actualizar, eliminar, listar, buscar, validar, bloquear), pero la clase UsuarioService solo implementa dos.

- Esto genera:
- Inconsistencia en la arquitectura.
- Diseño incompleto.
- Posible incumplimiento del principio de responsabilidad única.

Antipatrón: **SPAGHETTI CODE**

Se caracteriza por tener un diseño inconsistente, relaciones poco claras y una arquitectura poco definida.

3.4 Posible clase maestra en el Controlador

La clase UsuarioController contiene múltiples métodos relacionados con diferentes responsabilidades del usuario.

Si continúa creciendo, podría convertirse en una clase con demasiadas responsabilidades, dificultando:

- Mantenimiento.
- Escalabilidad.
- Comprensión del código.

Antipatrón: **APROPSCALYPSE**

Apropscalypse ocurre cuando una clase empieza a crecer descontroladamente en atributos y métodos, volviéndose difícil de mantener.

3.5 Ausencia de Interfaces (Falta de Abstracción)

No se utilizan interfaces como:

- IUsuarioService
- IUsuarioRepository

Esto dificulta:

- Aplicar correctamente el principio de inversión de dependencias.
- Implementar polimorfismo.
- Facilitar pruebas unitarias.

Antipatrón: **TESTER DRIVEN DEVELOPMENT**

Se caracteriza por no tener en cuenta las pruebas al momento de construir código, por lo que dificulta a futuro testear la funcionalidad y cumplimiento de requerimientos.